Programme: Computing and Digital Technologies

Student: Sofiya Serbina

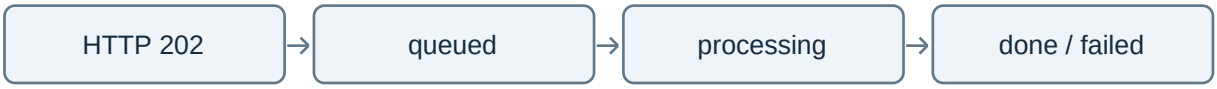
Laboratory Work No. 2

REST API and Message Queues - Remediation

Section A - Implementation and Results

The revised project separates a Flask HTTP API, local file storage and an independent worker. GET /time returns UTC server time. POST /echo returns a validated message. POST /jobs durably accepts text analysis and returns HTTP 202 with a UUID; it does not wait for analysis. GET /jobs/<job_id> returns the saved state and, when done, the result.

The worker converts text to uppercase and counts Unicode characters and whitespace-separated words. For example, the automated lifecycle test checks that "Hello queue" produces "HELLO QUEUE", 11 characters and 2 words. This performs real deterministic computation rather than a delay-only simulation.



Active record removed only after the terminal result has been saved.

Figure 1. Designed lifecycle and acknowledgement order (architecture illustration, not a screenshot).

A real loopback HTTP experiment was run on 30 October 2026. It used a temporary data directory and a separate worker process. The complete, unedited request output is in evidence/http-demo.txt. The experiment verified these outcomes:

Operation	Observed outcome
GET /time; valid POST /echo	HTTP 200, JSON responses
Invalid POST /echo; unknown job	HTTP 400 and HTTP 404
Three POST /jobs calls	HTTP 202 and distinct job IDs
GET while worker absent, after 30 s	All three jobs still queued
Start worker, then poll saved jobs	All three done; correct text results
Terminate worker with SIGTERM	Graceful process exit code 0

Table 1. Actual HTTP and process observations; detailed IDs, payloads and timestamps are in the supplied log.

Evidence limitation

Native Terminal access was denied by the authoring environment, so genuine terminal screenshots could not be captured. No synthetic screenshots were made. The supplied logs substantiate the executed experiments, but do not replace a teacher requirement for screenshots. screenshots/README.md lists the five captures to take manually before final resubmission.

Section B - Asynchronous Processing and Worker Recovery

Synchronous REST returns the result of the current request directly, as /echo does. With asynchronous communication, HTTP 202 confirms only acceptance. The queue retains work while the worker is stopped; clients later query status by job_id. The API and worker share a local data directory and coordinate short storage operations with file locks.

A claim atomically moves a queued file into processing. Writes use a temporary file, flush, fsync and atomic replacement. The worker saves a done or failed terminal record before removing its processing record. Deleting earlier would risk losing a task if the worker crashed. On restart, incomplete claims return to queued; a saved terminal result takes precedence over a leftover processing file.

The tests include a real child process killed after claiming a task. Restart recovered that task and completed it. A separate injected failure between terminal save and ACK verified that recovery preserves the already saved result without rewriting it. Corrupt records were quarantined while valid tasks continued; a second worker was refused. These checks cover process interruption, not all hardware or power-loss failures.

Measured acceptance checks

Check	Actual result	Threshold
Automated suite	41 passed; 0 failed	All tests pass
100 test-client submissions, p95	0.553 ms	< 1 second
10 jobs, worker absent	30.012 s; all retained	>= 30 s; 10/10
Drain after resume	0.0172 s; 10/10 correct	< 10 s; 10/10

Table 2. Executed local measurements. API timings use the Flask test client, not a network benchmark. Raw values: evidence/metrics.json; test output: evidence/pytest.txt.

Reflection and development history

FIFO selects by creation timestamp, with UUID ordering for ties. Job status makes acceptance, active processing and completion observable. Acknowledgement marks safe removal from the active queue; persistence keeps work across process restarts. Recovery provides at-least-once execution. Repeating deterministic text analysis is safe, but external side effects would require additional idempotency controls.

The original c5837d7 commit stored specification and code together and does not demonstrate specification-first development. This remediation uses an honest new sequence: c4772f7 updates only the specification, followed by 8652cab implementing the new architecture, then tests, documentation and evidence. Original history and the Lab 3 branch are retained.

The project is intended for one worker on one trusted local POSIX computer. It has no authentication, distributed storage or automatic retention policy. The remediation was prepared with OpenAI Codex assistance; the student must review the implementation and explain it at the defence. The evidence targets the rubric, but neither full screenshot credit nor a final mark of 25/25 is claimed.